

Homework – JavaScript Runtime and Async

Create one small JavaScript project to practice Closures, Call Stack, Promises, async/await, Event Loop, Tasks and Microtasks using HTML, CSS, and vanilla JavaScript.

The application should have several tasks, for example:

Load Users

Load Posts

Load Comments

Each task should have a name, status, execution count, and loading time.

Create tasks using a closure:

```
const task = createTask("Load Users");
```

```
task.run();
```

```
task.getCount();
```

```
task.reset();
```

The counter should be private. Different tasks must have their own counters.

Use `Promise` and `setTimeout` to simulate loading. Each task should take a random time, for example 500–2000 ms.

Some tasks should randomly fail. Show the result in the UI:

Load Users Completed

Load Posts Failed

Load Comments Completed

Add a 'Run All Task's button. Run all tasks at the same time and show "All tasks finished" only when all tasks have completed or failed.

Also compare sequential and concurrent execution:

```
await task1.run();
```

```
await task2.run();
```

```
await task3.run();
```

versus running all tasks together with a Promise method. Measure the execution time and explain why the results are different.

Add a Run Event Loop Demo button. Create your own example that contains:

```
console.log()
```

```
setTimeout()
```

```
Promise.resolve().then()
```

```
async / await
```

Your example should include at least two timers, two Promise callbacks and one async function.

Before running it, write your expected console output. Then run it and explain the real execution order using:

Call Stack → Microtask Queue → Task Queue → Event Loop

README

In README.md, briefly explain:

- How your closure keeps the task counter private.
- How the call stack works in one example from your application.
- How JavaScript can continue while `setTimeout` is waiting.
- Your predicted and actual Event Loop output.
- The difference between tasks and microtasks.
- How you handle multiple Promises and errors.
- The difference between sequential and concurrent execution.

Requirements

Use only HTML, CSS and vanilla JavaScript.